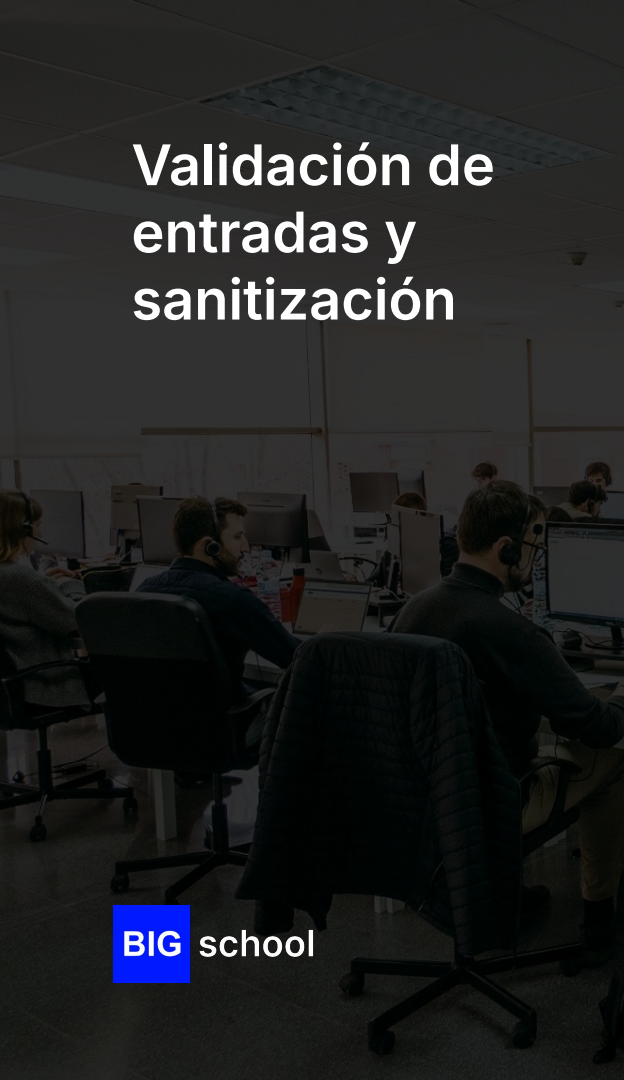


MÓDULO 8: SEGURIDAD

VALIDACIÓN DE ENTRADAS Y SANITIZACIÓN

PRÁCTICAS DE CODIFICACIÓN SEGURA

BIG school



Validación de entradas y sanitización

Índice

- Todo dato que entra puede ser peligroso
- Validar no es lo mismo que sanitizar
- Por qué fallamos los desarrolladores
- Dónde se valida
- Qué pasa si no lo haces
- Cómo piensan los sistemas modernos
- El papel de la IA en la validación
- Buenas prácticas integradas al diseño
- Integración con DevSecOps y pipelines
- Conclusión

Validación de entradas y sanitización

BIG school

BIGSEIO

GENERAMOS
NEGOCIO

Todo dato que entra puede ser peligroso

Nunca confíes en datos externos:
formularios, APIs o sistemas
internos.

Cualquier entrada puede ser
manipulada por error o ataque.

Validar no es lo mismo que sanitizar

- Validar: comprobar que el dato cumple con formato y valores esperados.
- Sanitizar: limpiar datos para eliminar o neutralizar contenido peligroso.

Ambas prácticas se complementan: primero validar, luego hacer seguro.

Por qué fallamos los desarrolladores

- Confiar solo en validación frontend.
- Asumir confiabilidad de sistemas internos.
- Usar validaciones genéricas o inapropiadas.
- Asumir sin validar que el dato “ya venía bien”.

Dónde se valida

1. **En todas las capas: frontend, backend, APIs y base de datos.**
2. **Cada microservicio debe asumir datos no confiables.**
3. **Aplica principio Zero Trust: confía en nada, valida todo.**

Qué pasa si no lo haces

Las vulnerabilidades comienzan aquí:
inyecciones, XSS, directorios, fugas.

Permites a atacantes controlar el flujo y
romper el sistema.

Cómo piensan los sistemas modernos

- Frameworks modernos integran validación y sanitización automática.
- Pero nunca delegues totalmente: el criterio humano sigue siendo clave

El papel de la IA en la validación

- La IA ayuda a generar validaciones y detectar riesgos.
- Puede generar validaciones incorrectas: usa IA como apoyo, no sustituto.

Buenas prácticas integradas al diseño

- Validación y sanitización parte del Security by Design y Default.
- Preguntar desde el diseño: qué datos, formatos y riesgos existen.
- Definir validación desde el principio para robustez.

Integración con DevSecOps y pipelines

- Validación continua en pipelines con herramientas automáticas.
- Feedback constante y mejora proactiva, no solo auditorías tardías.

Conclusión

La validación es clave para evitar fallos graves.

Un sistema seguro dice “no” a entrada inesperada.

Esa es la diferencia entre programar y construir software seguro.